3.1 System Architecture

Selected Architecture: Layered Architecture

The Moderated Learning Management System will use a four-layer architecture consisting of the Presentation Layer, Business Logic Layer, Data Access Layer, and Database Layer.

The **Presentation Layer** provides a simple console interface through which students, lecturers, Heads of Department (HoDs), and administrators interact with the system.

The **Business Logic Layer** implements the system's rules, including authentication, role-based access control, assignment submissions, deadline extensions, late penalties, marking, moderation, mark release, and audit logging.

The **Data Access Layer** manages communication with the database through repositories or Data Access Objects (DAOs). It performs database operations without requiring the presentation layer to understand SQL.

The **Database Layer** uses SQLite to store user accounts, departments, courses, enrolments, assignments, submissions, marks, feedback, and audit records.

Justification

Layered Architecture was selected because it provides a clear separation of responsibilities and is suitable for a small C++17 application using SQLite. It supports offline operation, makes the system easier to maintain and test, and helps ensure that business rules are enforced consistently regardless of which user interacts with the system.

The architecture also supports future database changes because database-specific operations are isolated in the Data Access Layer. The simple console interface supports the requirement that teachers should not need advanced IT skills.

Rejected Alternatives:

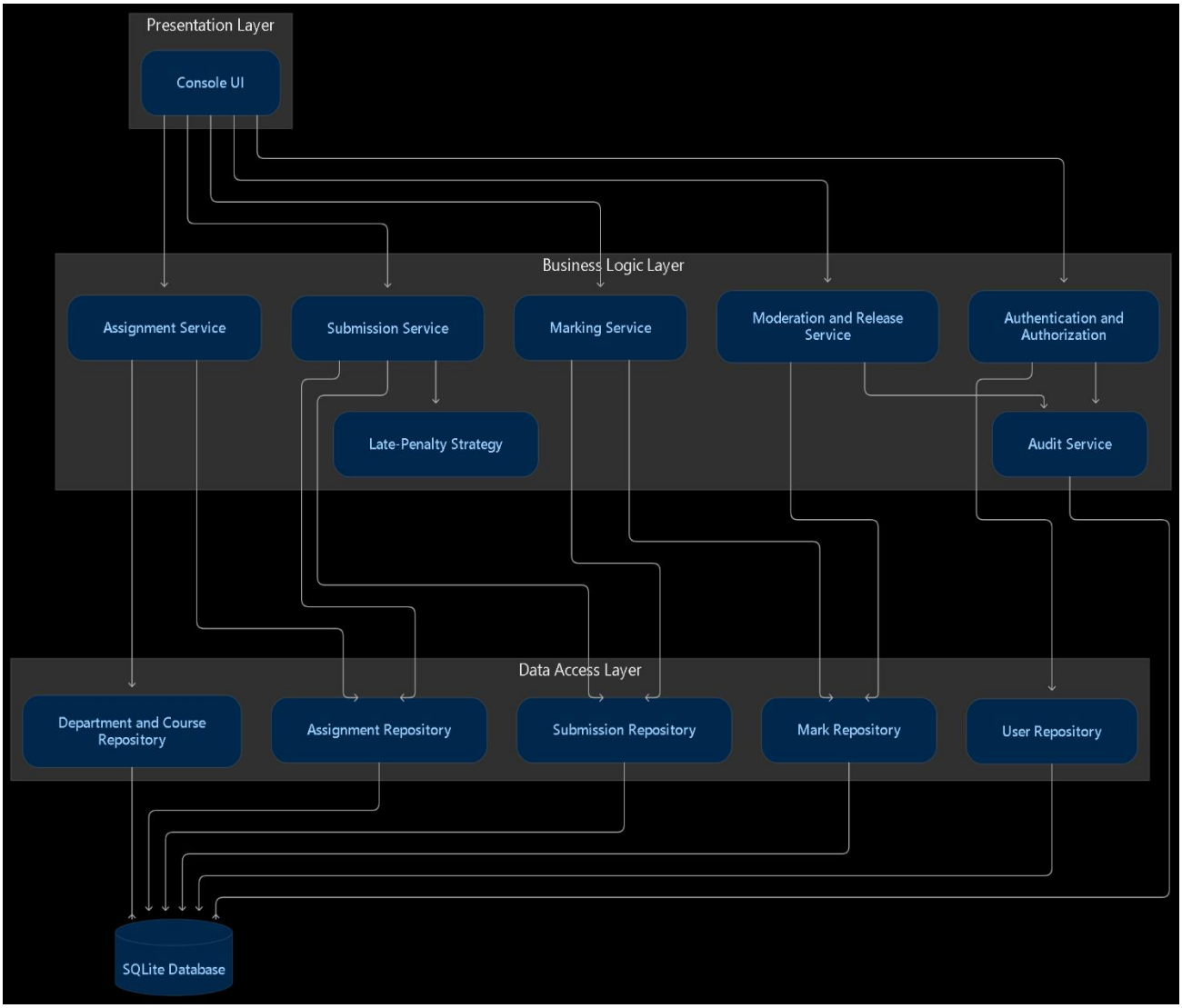
Model-View-Controller (MVC)

MVC separates an application into Model, View, and Controller components. Although it can be useful for applications with complex user interfaces, its additional separation is not necessary for the current console-based system. A four-layer design provides sufficient separation while remaining straightforward to implement and understand. MVC could be reconsidered if the system later develops a more complex graphical or web interface.

Client-Server / Microservices Architecture

The system is specified to run locally on a single school computer or local SQLite file without requiring an external database server. Introducing a client-server network architecture would add unnecessary network latency, deployment complexity, and configuration overhead, directly violating the Principal's simplicity constraint for non-IT expert teachers.

Component Diagram



Console UI: accepts user input and displays results.

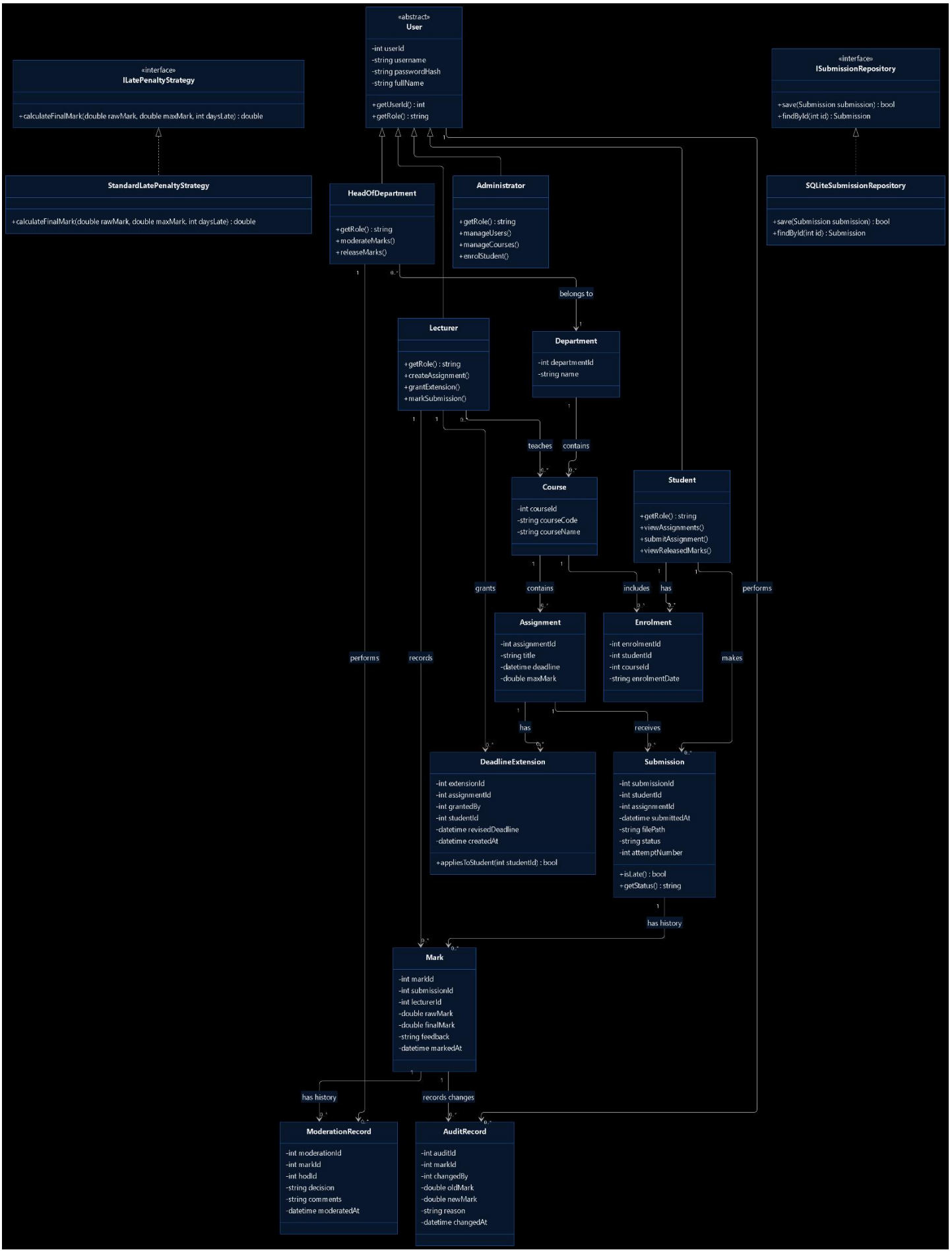
Business logic components: enforce system rules, including permissions, deadlines, penalties, marking, moderation, and release.

Repository components: retrieve and save information without exposing SQL details to the business logic.

SQLite Database: stores the persistent system data.

Late-Penalty Strategy: calculates the penalty according to the configured policy.

3.2 Class diagram and design patterns


«abstract»
User
-int userId
-string username
-string passwordHash
-string fullName
+getUserId() : int
+getRole() : string
«interface»
ILatePenaltyStrategy
+calculateFinalMark(double rawMark, double maxMark, int daysLate) : double
«interface»
ISubmissionRepository
+save(Submission submission) : bool
+findById(int id) : Submission
StandardLatePenaltyStrategy
+calculateFinalMark(double rawMark, double maxMark, int daysLate) : double
SQLiteSubmissionRepository
+save(Submission submission) : bool
+findById(int id) : Submission
HeadOfDepartment
+getRole() : string
+moderateMarks()
+releaseMarks()
Administrator
+getRole() : string
+manageUsers()
+manageCourses()
+enrolStudent()
belongs to
Lecturer
+getRole() : string
+createAssignment()
+grantExtension()
+markSubmission()
Department
-int departmentId
-string name
teaches
contains
Course
-int courseId
-string courseCode
-string courseName
Student
+getRole() : string
+viewAssignments()
+submitAssignment()
+viewReleasedMarks()
grants
contains
includes
has
performs
Assignment
-int assignmentId
-string title
-datetime deadline
-double maxMark
Enrolment
-int enrolmentId
-int studentId
-int courseId
-string enrolmentDate
performs
records
makes
has
receives
DeadlineExtension
-int extensionId
-int assignmentId
-int grantedBy
-int studentId
-datetime revisedDeadline
-datetime createdAt
+appliesToStudent(int studentId) : bool
Submission
-int submissionId
-int studentId
-int assignmentId
-datetime submittedAt
-string filePath
-string status
-int attemptNumber
+isLate() : bool
+getStatus() : string
has history
Mark
-int markId
-int submissionId
-int lecturerId
-double rawMark
-double finalMark
-string feedback
-datetime markedAt
has history
records changes
ModerationRecord
-int moderationId
-int markId
-int hodId
-string decision
-string comments
-datetime moderatedAt
AuditRecord
-int auditId
-int markId
-int changedBy
-double oldMark
-double newMark
-string reason
-datetime changedAt

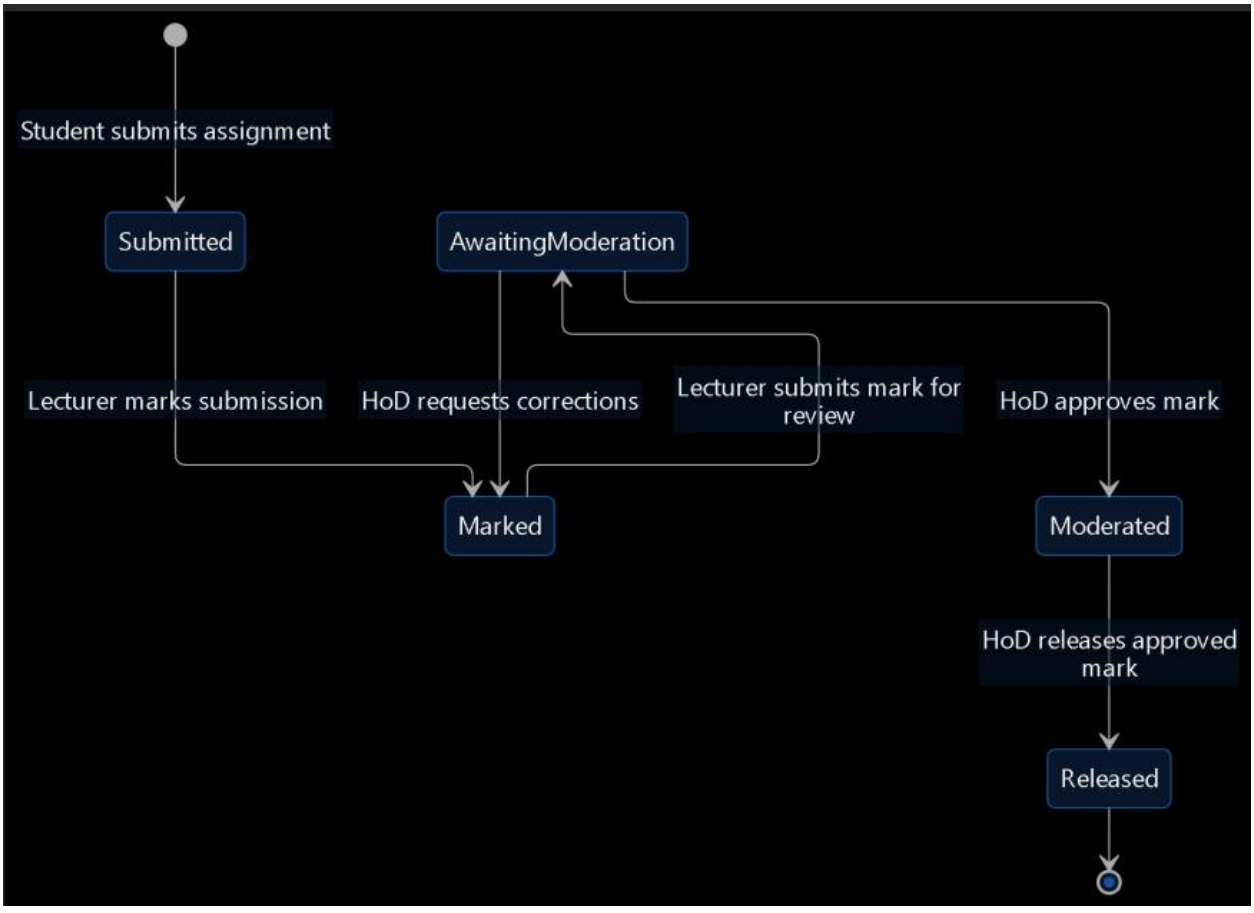
Design Patterns and SOLID Principles

Strategy Pattern: The system uses the ILatePenaltyStrategy interface and StandardLatePenaltyStrategy implementation to encapsulate the late-penalty calculation. This makes the policy independently testable and replaceable without coupling the submission workflow to one penalty algorithm.

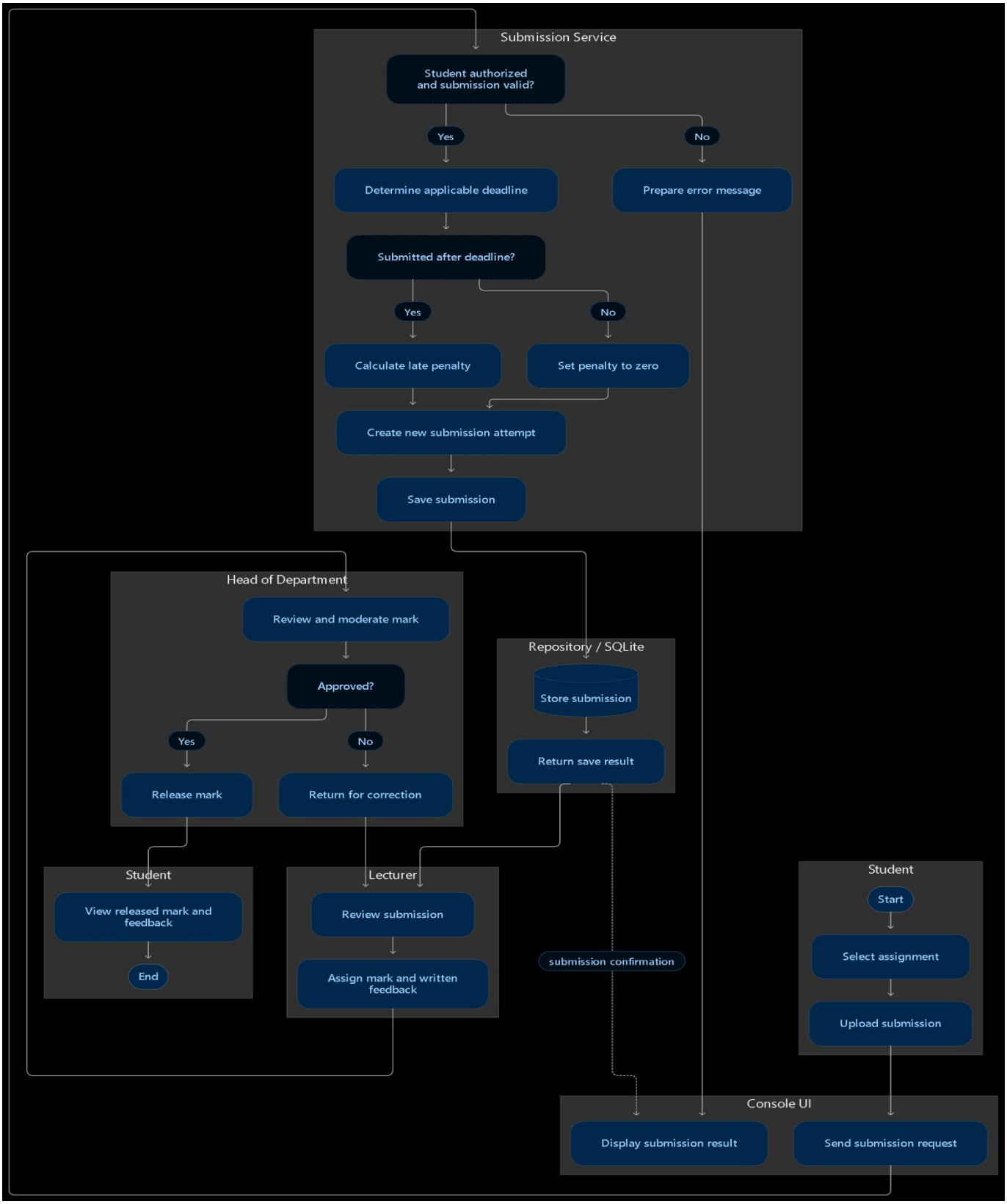
Repository Pattern: The ISubmissionRepository interface separates submission persistence from business logic. SQLiteSubmissionRepository implements the interface and handles SQLite operations. This improves maintainability and supports testing without requiring the business logic to execute SQL directly.

The design applies SOLID principles through focused class responsibilities, replaceable penalty strategies, substitutable user subclasses, small interfaces, and dependencies on abstractions. These decisions support maintainability and testability while keeping the implementation appropriate for a C++17 application using SQLite.

3.3 Behavioural models: State Machine

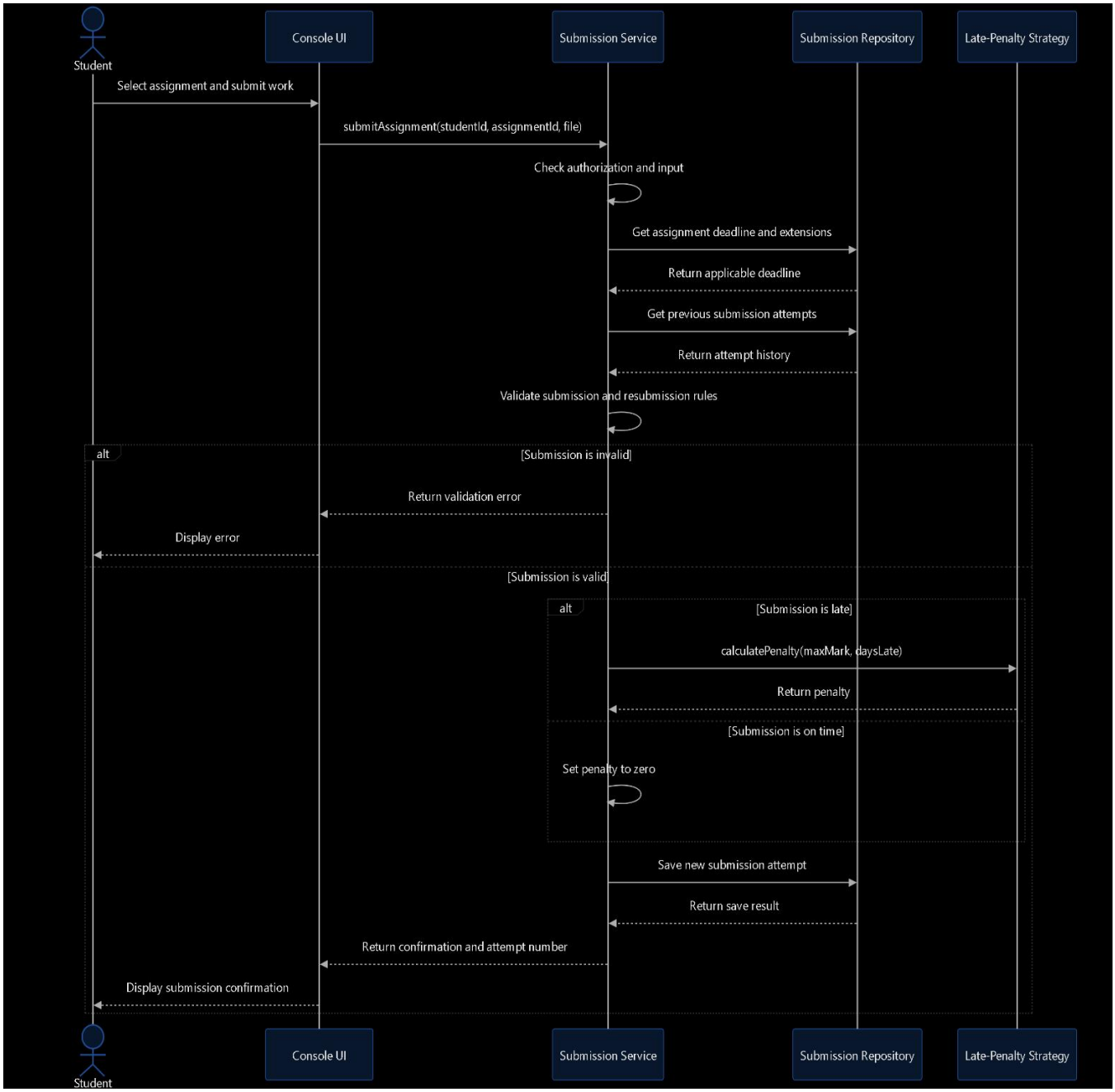


Activity Diagram with Swimlanes



Sequence Diagrams

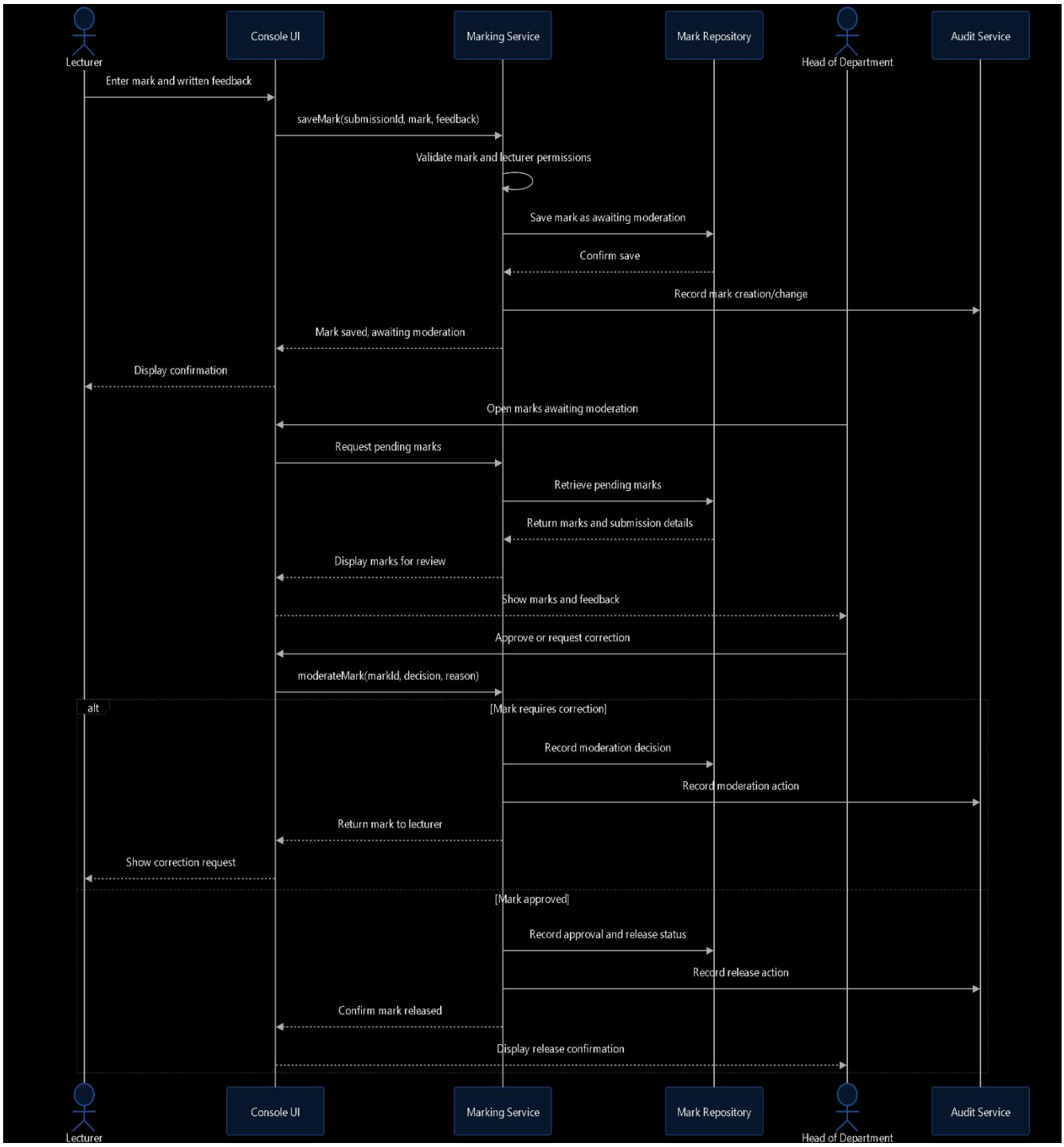
i) Submit Assignment



Late-penalty rule: 10% of the assignment's maximum mark per day late, for up to three days; work more than three days late receives zero under the stated policy. An approved extension must be applied before calculating lateness.

The strategy calculates the penalty; the submission service remains responsible for applying the rules and coordinating the process.

ii) Moderate and Release Marks

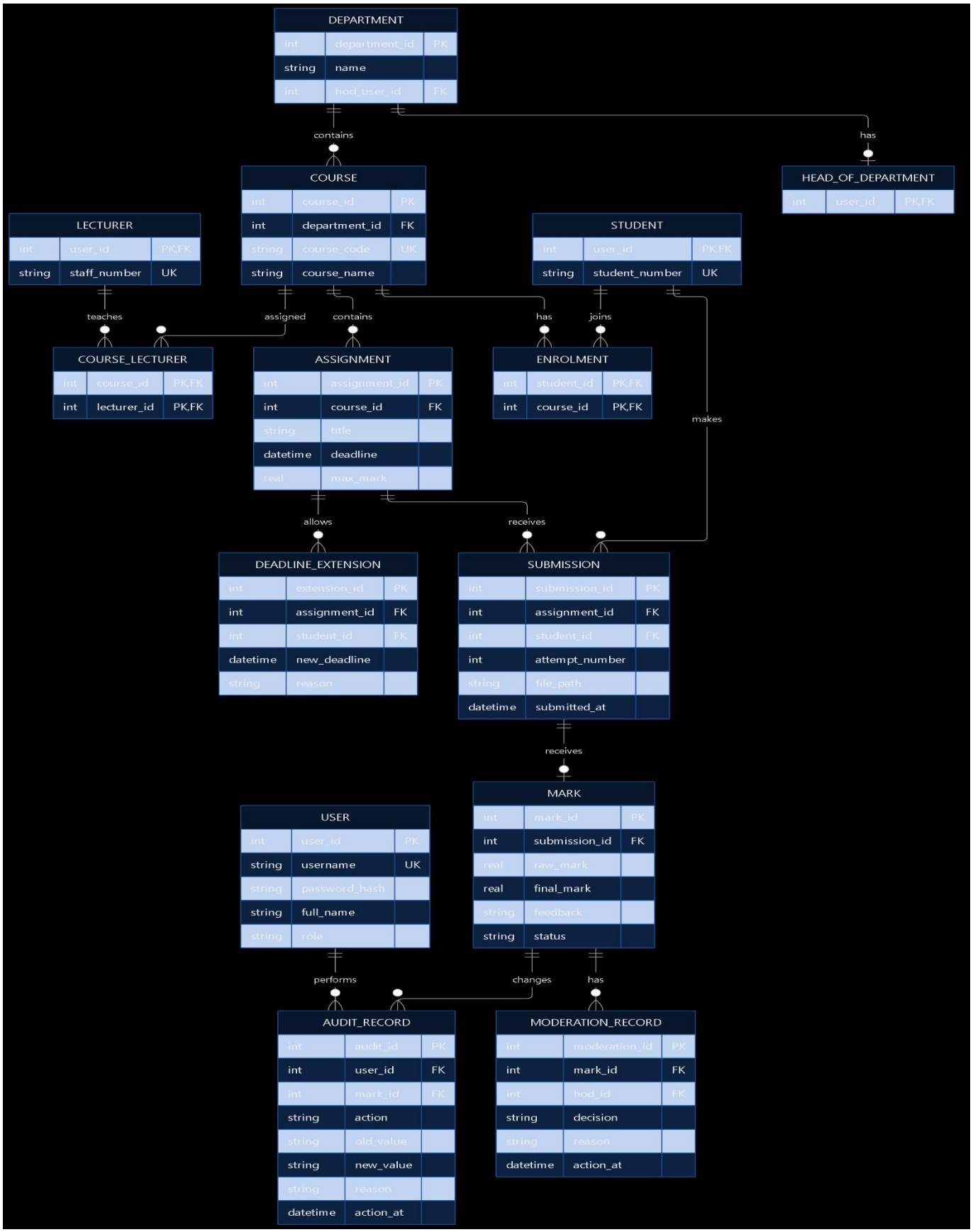


3.4 Database Design

ERD

ERD assumptions: USER stores common account information; role-specific tables store additional information. Each submission belongs to one student and one assignment. Each submission has at most one current mark, while moderation and audit records preserve the history of decisions and changes.

A whole-class extension has a NULL student_id; an individual extension has a specific student ID. The application must validate the extension's scope and use the correct deadline.


DEPARTMENT
int department_id PK
string name
int hod_user_id FK
contains
has
HEAD_OF_DEPARTMENT
int user_id PK,FK
COURSE
int course_id PK
int department_id FK
string course_code UK
string course_name
LECTURER
int user_id PK,FK
string staff_number UK
STUDENT
int user_id PK,FK
string student_number UK
teaches
assigned
contains
has
joins
COURSE_LECTURER
int course_id PK,FK
int lecturer_id PK,FK
ASSIGNMENT
int assignment_id PK
int course_id FK
string title
datetime deadline
real max_mark
ENROLMENT
int student_id PK,FK
int course_id PK,FK
makes
allows
receives
DEADLINE_EXTENSION
int extension_id PK
int assignment_id FK
int student_id FK
datetime new_deadline
string reason
SUBMISSION
int submission_id PK
int assignment_id FK
int student_id FK
int attempt_number
string file_path
datetime submitted_at
receives
MARK
int mark_id PK
int submission_id FK
real raw_mark
real final_mark
string feedback
string status
USER
int user_id PK
string username UK
string password_hash
string full_name
string role
performs
changes
has
AUDIT_RECORD
int audit_id PK
int user_id FK
int mark_id FK
string action
string old_value
string new_value
string reason
datetime action_at
MODERATION_RECORD
int moderation_id PK
int mark_id FK
int hod_id FK
string decision
string reason
datetime action_at

SQL Schema

```
PRAGMA foreign_keys = ON;

CREATE TABLE User (
    user_id INTEGER PRIMARY KEY,
    username TEXT NOT NULL UNIQUE,
    password_hash TEXT NOT NULL,
    full_name TEXT NOT NULL,
    role TEXT NOT NULL CHECK (
        role IN ('Student', 'Lecturer', 'HoD', 'Administrator')
    )
);

CREATE TABLE Department (
    department_id INTEGER PRIMARY KEY,
    name TEXT NOT NULL UNIQUE,
    hod_user_id INTEGER,
    FOREIGN KEY (hod_user_id) REFERENCES User(user_id)
);

CREATE TABLE Student (
    user_id INTEGER PRIMARY KEY,
    student_number TEXT NOT NULL UNIQUE,
    FOREIGN KEY (user_id) REFERENCES User(user_id)
);

CREATE TABLE Lecturer (
    user_id INTEGER PRIMARY KEY,
```

```
    staff_number TEXT NOT NULL UNIQUE,
    FOREIGN KEY (user_id) REFERENCES User(user_id)
);

CREATE TABLE HeadOfDepartment (
    user_id INTEGER PRIMARY KEY,
    FOREIGN KEY (user_id) REFERENCES User(user_id)
);

CREATE TABLE Course (
    course_id INTEGER PRIMARY KEY,
    department_id INTEGER NOT NULL,
    course_code TEXT NOT NULL UNIQUE,
    course_name TEXT NOT NULL,
    FOREIGN KEY (department_id) REFERENCES Department(department_id)
);

CREATE TABLE CourseLecturer (
    course_id INTEGER NOT NULL,
    lecturer_id INTEGER NOT NULL,
    PRIMARY KEY (course_id, lecturer_id),
    FOREIGN KEY (course_id) REFERENCES Course(course_id),
    FOREIGN KEY (lecturer_id) REFERENCES Lecturer(user_id)
);

CREATE TABLE Enrolment (
    student_id INTEGER NOT NULL,
    course_id INTEGER NOT NULL,
    PRIMARY KEY (student_id, course_id),
```

```
    FOREIGN KEY (student_id) REFERENCES Student(user_id),
    FOREIGN KEY (course_id) REFERENCES Course(course_id)
);

CREATE TABLE Assignment (
    assignment_id INTEGER PRIMARY KEY,
    course_id INTEGER NOT NULL,
    title TEXT NOT NULL,
    description TEXT,
    deadline TEXT NOT NULL,
    max_mark REAL NOT NULL CHECK (max_mark > 0),
    FOREIGN KEY (course_id) REFERENCES Course(course_id)
);

CREATE TABLE DeadlineExtension (
    extension_id INTEGER PRIMARY KEY,
    assignment_id INTEGER NOT NULL,
    student_id INTEGER,
    new_deadline TEXT NOT NULL,
    reason TEXT NOT NULL,
    granted_by INTEGER NOT NULL,
    FOREIGN KEY (assignment_id) REFERENCES Assignment(assignment_id),
    FOREIGN KEY (student_id) REFERENCES Student(user_id),
    FOREIGN KEY (granted_by) REFERENCES Lecturer(user_id)
);

CREATE TABLE Submission (
    submission_id INTEGER PRIMARY KEY,
    assignment_id INTEGER NOT NULL,
```

```
    student_id INTEGER NOT NULL,
    attempt_number INTEGER NOT NULL CHECK (attempt_number > 0),
    file_path TEXT NOT NULL,
    submitted_at TEXT NOT NULL,
    UNIQUE (assignment_id, student_id, attempt_number),
    FOREIGN KEY (assignment_id) REFERENCES Assignment(assignment_id),
    FOREIGN KEY (student_id) REFERENCES Student(user_id)
);

CREATE TABLE Mark (
    mark_id INTEGER PRIMARY KEY,
    submission_id INTEGER NOT NULL UNIQUE,
    raw_mark REAL NOT NULL CHECK (raw_mark >= 0),
    final_mark REAL NOT NULL CHECK (final_mark >= 0),
    feedback TEXT NOT NULL,
    status TEXT NOT NULL CHECK (
        status IN ('AwaitingModeration', 'CorrectionRequired',
                'Approved', 'Released')
    ),
    marked_by INTEGER NOT NULL,
    marked_at TEXT NOT NULL,
    FOREIGN KEY (submission_id) REFERENCES Submission(submission_id),
    FOREIGN KEY (marked_by) REFERENCES Lecturer(user_id)
);

CREATE TABLE ModerationRecord (
    moderation_id INTEGER PRIMARY KEY,
    mark_id INTEGER NOT NULL,
    hod_id INTEGER NOT NULL,
```

```
    decision TEXT NOT NULL CHECK (
        decision IN ('Approved', 'CorrectionRequired')
    ),
    reason TEXT,
    action_at TEXT NOT NULL,
    FOREIGN KEY (mark_id) REFERENCES Mark(mark_id),
    FOREIGN KEY (hod_id) REFERENCES HeadOfDepartment(user_id)
);

CREATE TABLE AuditRecord (
    audit_id INTEGER PRIMARY KEY,
    user_id INTEGER NOT NULL,
    mark_id INTEGER,
    action TEXT NOT NULL,
    old_value TEXT,
    new_value TEXT,
    reason TEXT,
    action_at TEXT NOT NULL,
    FOREIGN KEY (user_id) REFERENCES User(user_id),
    FOREIGN KEY (mark_id) REFERENCES Mark(mark_id)
);
```

Table Dictionary

Table	Purpose	Important fields
User	Common account and login details	user_id, username, password_hash, role
Department	Academic departments	department_id, hod_user_id
Student	Student-specific details	user_id, student_number
Lecturer	Lecturer-specific details	user_id, staff_number
HeadOfDepartment	HoD role identity	user_id
Course	Courses offered by departments	course_id, course_code
CourseLecturer	Lecturer-course assignments	course_id, lecturer_id
Enrolment	Student-course relationships	student_id, course_id
Assignment	Assignment details and deadlines	assignment_id, deadline, max_mark
DeadlineExtension	Individual and whole-class extensions	assignment_id, nullable student_id, new_deadline
Submission	Each submitted attempt	attempt_number, submitted_at, file_path
Mark	Current mark and feedback for a submission	raw_mark, final_mark, status
ModerationRecord	HoD moderation decisions	decision, reason, action_at
AuditRecord	Record of mark-related actions	old_value, new_value, user_id, action_at

Normalization to Third Normal Form (3NF)

The proposed database is designed around these principles:

1. First Normal Form (1NF): Each field stores a single value. Multiple submissions are stored as separate rows, not as lists in one field.
2. Second Normal Form (2NF): In tables with composite keys, such as Enrolment and CourseLecturer, non-key attributes depend on the entire key.
3. Third Normal Form (3NF): Student, lecturer, course, and department details are stored in their own tables rather than repeatedly copied into assignments or submissions. Role-specific details are separated from common user details.

This reduces duplication and helps maintain consistency when details change.

Class-to-table mapping

Design class	Database table(s)	Reason
User	User	Common account details
Student	Student	Student-specific details
Lecturer	Lecturer	Lecturer-specific details
HeadOfDepartment	HeadOfDepartment	HoD identity and role
Department	Department	Department details
Course	Course, CourseLecturer	Course data and lecturer assignments
Enrolment	Enrolment	Many-to-many student-course relationship
Assignment	Assignment	Assignment information
DeadlineExtension	DeadlineExtension	Extension scope and new deadline
Submission	Submission	One row per attempt
Mark	Mark	Current mark, feedback, and status
ModerationRecord	ModerationRecord	Moderation history
AuditRecord	AuditRecord	Audit trail
ILatePenaltyStrategy and StandardLatePenaltyStrategy	No direct table	Business logic; calculate penalties
ISubmissionRepository and SQLiteSubmissionRepository	No direct table	Interfaces and data-access implementation